Lab 14 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — From lab 08 to the cluster

Answer. `ENV` → stays in the image (a default value, unchanged). `-e` → `env:` with a hard-coded `value` in the template. `--env-file api.env` → a ConfigMap (`--from-env-file`) injected with `envFrom`. Bind-mounted `application.yaml` → a `--from-file` ConfigMap mounted as a volume in `/config/`. `podman secret` + `_FILE` → a Secret mounted as a `secret:` volume with `DB_PASSWORD_FILE`. No equivalent: the `-e` **typed on the command line** — and that is a good thing: in the cluster, every piece of configuration is a versioned, re-readable object, not an argument lost in the shell history.

Why. The cluster has no command line: every value must exist in the API to be applied on any node.

Nuance. `kubectl set env deployment/api APP_PROFILE=prod` exists and hard-codes the value into the template — handy for a test, to be avoided in production (the value is no longer in Git).

Example.

```
kubectl create configmap api-config --from-env-file=api.env --dry-run=client -o yaml
```

Question 2 — Frozen variables, live files

Answer. First `grep`: `APP_MESSAGE=bonjour depuis une ConfigMap`, **the old value** — variables are copied into the process at startup. Second `cat` (after a minute): `<h1>modifie</h1>` — the volume is updated by the kubelet. For the API to see the new message: `kubectl rollout restart deployment/api`, which recreates the Pods.

Why. An environment variable belongs to the process; a ConfigMap volume is a directory the kubelet resyncs periodically (`..data` symlink swapped atomically).

Nuance. The volume update delay depends on the kubelet's sync period (one minute by default) and on caching; allow one to two minutes. And nginx loaded `default.conf` at startup: a new configuration file requires `nginx -s reload`, a new `index.html` is served immediately.

Example.

```
kubectl exec deploy/web -- ls -l /usr/share/nginx/html/     # index.html -> ..data/index.html
kubectl rollout restart deployment/api && kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Question 3 — `CreateContainerConfigError`

Answer. `kubectl describe pod api-5dff955d5b-rz7sd` shows in the Events: `Error: couldn't find key profil in ConfigMap lab14/api-config` — the referenced key does not exist (`profil` instead of `APP_PROFILE`). The kubelet cannot build the container's environment, so it does not start it. The old Pod is still running because the rollout waits for the new one to be ready before removing the old (lab 12): the service is untouched. Fix: repair the key (or `rollout undo`).

Why. References to ConfigMaps and Secrets are resolved **when the container is created**, not at `apply`: a manifest can be accepted and fail later.

Nuance. `optional: true` on the reference (`configMapKeyRef` or `configMapRef`) ignores a missing key or object instead of blocking — only for genuinely optional configuration.

Example.

```
kubectl describe pod -l app=api | grep -A1 "couldn't find key"
kubectl rollout undo deployment/api
```

Question 4 — "Encrypted, therefore safe"

Answer. `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` prints `S3cr3t-pg`: base64 is an **encoding**, reversible by anyone. What a Secret really brings: (1) a **distinct object class**, which RBAC can forbid reading separately from ConfigMaps; (2) a **tmpfs** mount on the node, never written to its disk; (3) **masking** in `describe` and in tools, which prevents leaks by copy-paste. The three enterprise measures: encryption at rest in etcd (`EncryptionConfiguration` or KMS), strict RBAC on `get`/`list secrets`, and no plain-text Secret in Git (External Secrets, Sealed Secrets, SOPS).

Why. Base64 only exists to carry arbitrary bytes inside JSON/YAML; it never had a protective role.

Nuance. Even encrypted at rest, a Secret is **plain text** to anyone allowed to read it through the API: RBAC is the real boundary.

Example.

```
kubectl get secret db-secret -o yaml | grep POSTGRES_PASSWORD     # UzNjcjN0LXBn
echo UzNjcjN0LXBn | base64 -d                                      # S3cr3t-pg
```

Question 5 — Variable or file for the password

Answer. Visibility: as a variable, `kubectl exec deploy/api -- env` prints the password (and so does any environment dump); as a file, only the path is visible, the value requires opening the file. Rotation: as a variable, a new Secret is only seen after `rollout restart`; as a file, the kubelet updates it within a minute, and an application that re-reads the file (or a pool that reconnects with the new password) follows without a restart. Spring Boot: both work, the second through `spring.config.import=optional:configtree:/run/secrets/db/`. The **file** is chosen, as in lab 08.

Why. The environment is widely exposed metadata; a mounted file is reachable only by the process.

Nuance. Many official images accept only the variable (`POSTGRES_PASSWORD`) **or** the file (`POSTGRES_PASSWORD_FILE`): check the image's documentation.

Example.

```
kubectl exec deploy/api -- env | grep -c S3cr3t        # 0 with the volume
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD
```

Question 6 — `subPath` does not update

Answer. With `subPath`, the kubelet **copies** the file into the container at startup instead of mounting the symlinked directory: ConfigMap updates are never propagated. Two ways out: (1) mount the ConfigMap on a **dedicated directory** (the whole `/etc/nginx/conf.d/`, or `/etc/nginx/extra/` included from the main configuration) without `subPath`; (2) accept the `subPath` and trigger a `rollout restart` on every change — or change the ConfigMap's name (question 12), which modifies the template and causes the rollout.

Why. `subPath` exists to inject one file into a directory that holds other things; the price is losing dynamic updates.

Nuance. The same trap applies to Secrets mounted with `subPath`: rotation is not seen.

Example.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' | grep -o 'subPath[^}]*'
```

Question 7 — Same Deployment, another namespace

Answer. The Deployment is created in `prod`, its Pods start with `APP_PROFILE=prod` and the production password: references resolve **within the Pod's namespace**, to the `api-config` and `db-secret` objects found there. Nothing in the manifest changed. This demonstrates that promoting an image (lab 10) is an `apply` of the same file in another namespace (or cluster), where the configuration differs: the tested artefact is the deployed artefact.

Why. References by name are local to the namespace; that is the boundary that carries configuration.

Nuance. If `api-config` is missing in `prod`, the Pods stay in `CreateContainerConfigError` (question 3): the ConfigMap is part of what must be deployed, and Helm/Kustomize (lab 17) ship it with the rest.

Example.

```
kubectl apply -f deployment-api.yaml -n prod && kubectl exec -n prod deploy/api -- env | grep APP_PROFILE   # prod
```

Question 8 — `--dry-run=client -o yaml`

Answer. The command **sends nothing** to the cluster: it prints the complete `Secret` manifest, with `data.password` already base64-encoded (`eA==` for `x`), ready to be saved to a file. It is used to get the exact structure without getting the encoding or indentation wrong. What must not be done: commit that file as is — it contains the secret, base64 or not. It must be encrypted (SOPS, Sealed Secrets) before entering Git, or serve as a template with no value.

Why. `--dry-run=client` makes `kubectl` do all the object-building work, with no API call.

Nuance. `--dry-run=server` sends the object for validation without persisting it: useful for testing the cluster's webhooks and rules.

Example.

```
kubectl create secret generic db --from-literal=password=x --dry-run=client -o yaml | grep -A1
'^data:'   # password: eA==
```

Question 9 — Secret pasted into a ticket

Answer. (1) The Secret is **compromised**: base64 or not, it has been read. (2) **Rotation**: new password on the database side, `kubectl apply` (or the secrets manager) of an updated `db-secret`. (3) **Propagation**: if the Secret is mounted as a volume, the Pods see the new file within a minute, and an application that re-reads the file follows; as a variable, `rollout restart deployment/api` is mandatory. (4) Revocation of the old password, check of the database logs, ticket clean-up last. What makes rotation fast: a Secret referenced by name, mounted as a file, read by an application that can reconnect; slow: a value copied into several manifests, as a variable, with Pods to restart one by one.

Why. The only safe answer remains making the leaked value useless.

Nuance. The RBAC of lab 16 reduces the number of people able to produce that `get -o yaml`: that is the preventive measure.

Example.

```
kubectl create secret generic db-secret --from-literal=POSTGRES_PASSWORD=new --dry-run=client -
o yaml | kubectl apply -f -
kubectl rollout restart deployment/api
```

Question 10 — An empty directory

Answer. The **`volumeMounts`** is missing (or its `mountPath` does not target `/usr/share/nginx/html`): the ConfigMap may be declared under `volumes:`, but nothing mounts it into the container — or the volume is mounted at another path. Check: `kubectl get pod -l app=web -o yaml` and compare `spec.volumes` (the `html` volume with `configMap: web-html`) with `spec.containers[0].volumeMounts` (an `html` mount on `/usr/share/nginx/html`). An empty directory (rather than the image's original `index.html`) indicates that a volume **is** mounted there, but not the right one: for instance an empty ConfigMap or one with another name.

Why. `volumes` declares, `volumeMounts` connects; both are needed, and a mount hides what the image contained.

Nuance. `kubectl describe pod` shows `Mounts:` and `Volumes:` readably, and a volume whose ConfigMap does not exist blocks in `ContainerCreating` with `configmap "web-html" not found`.

Example.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' ; echo
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.volumes}' ; echo
```

Question 11 — Mounting the whole `/etc/nginx/conf.d`

Answer. Mounting the whole directory **hides** the image's content at that location: the original `default.conf` disappears, replaced by the ConfigMap's single file — intended, since that is the file being replaced, and it keeps dynamic updates (no `subPath`). To add a second `.conf` file: put it in

the **same** ConfigMap (one more key, one more file in the directory), or use `projected` to combine several ConfigMaps in one mount.

Why. A mount replaces a directory, exactly like a bind mount in lab 06; the image underneath is not merged.

Nuance. If the image expected other files in that directory (not the case here), they would have to be reproduced in the ConfigMap or mounted elsewhere.

Example.

```
kubectl exec deploy/web -- ls /etc/nginx/conf.d/     # default.conf only, from the ConfigMap
```

Question 12 — `immutable` or a versioned name

Answer. `immutable: true` forbids any modification of the object: protection against an accidental edit in production, and the kubelet stops watching it (less load on the API server) — but changing the configuration requires **creating another object** and changing the reference. Naming it `api-config-v2` and pointing the Deployment at it changes the **template**, hence triggers a **rollout**: the Pods restart with the new configuration, and `rollout undo` brings the old one back (the old ReplicaSet references `api-config-v1`). That is why Kustomize (`configMapGenerator`) and Helm (checksum annotation) add a hash suffix: every configuration change becomes a Deployment revision, traceable and reversible.

Why. A ConfigMap modified in place is not a revision; a renamed ConfigMap is.

Nuance. Old suffixed ConfigMaps accumulate; Kustomize and Helm clean them up (or not) depending on configuration.

Example.

```
kubectl patch configmap api-config -p '{"immutable":true}' && kubectl patch configmap api-config -p '{"data":{"APP_PROFILE":"x"}}'   # field is immutable
```

Question 13 — No Secret in Git

Answer. The repository holds the Deployment (which references `db-secret` by name) and an **ExternalSecret** object: "create a Secret `db-secret` with the key `POSTGRES_PASSWORD` taken from Vault at path `secret/app/prod/db`". No value. In the cluster, the **External Secrets Operator** reads that object, authenticates with the secrets manager (Vault, AWS Secrets Manager…), fetches the value and **creates or updates** the Kubernetes Secret — and refreshes it periodically, which performs the rotation. With Sealed Secrets, the repository holds the Secret **encrypted** with the cluster's public key, and a controller decrypts it into an ordinary Secret. The link with lab 08: the secrets manager is the source of truth, the Kubernetes Secret is only a local cache injected into the Pods.

Why. Git is a distribution medium; the cluster is where a secret can live under access control.

Nuance. Both approaches keep the Secret **in plain text in the API** once created: encryption at rest and RBAC remain necessary.

Example.

```
kind: ExternalSecret
spec:
  secretStoreRef: { name: vault }
  target: { name: db-secret }
  data:
    - secretKey: POSTGRES_PASSWORD
      remoteRef: { key: secret/app/prod/db, property: password }
```